



SAIR

Spatial AI & Robotics Lab

CSE 473/573

L10: STITCHING

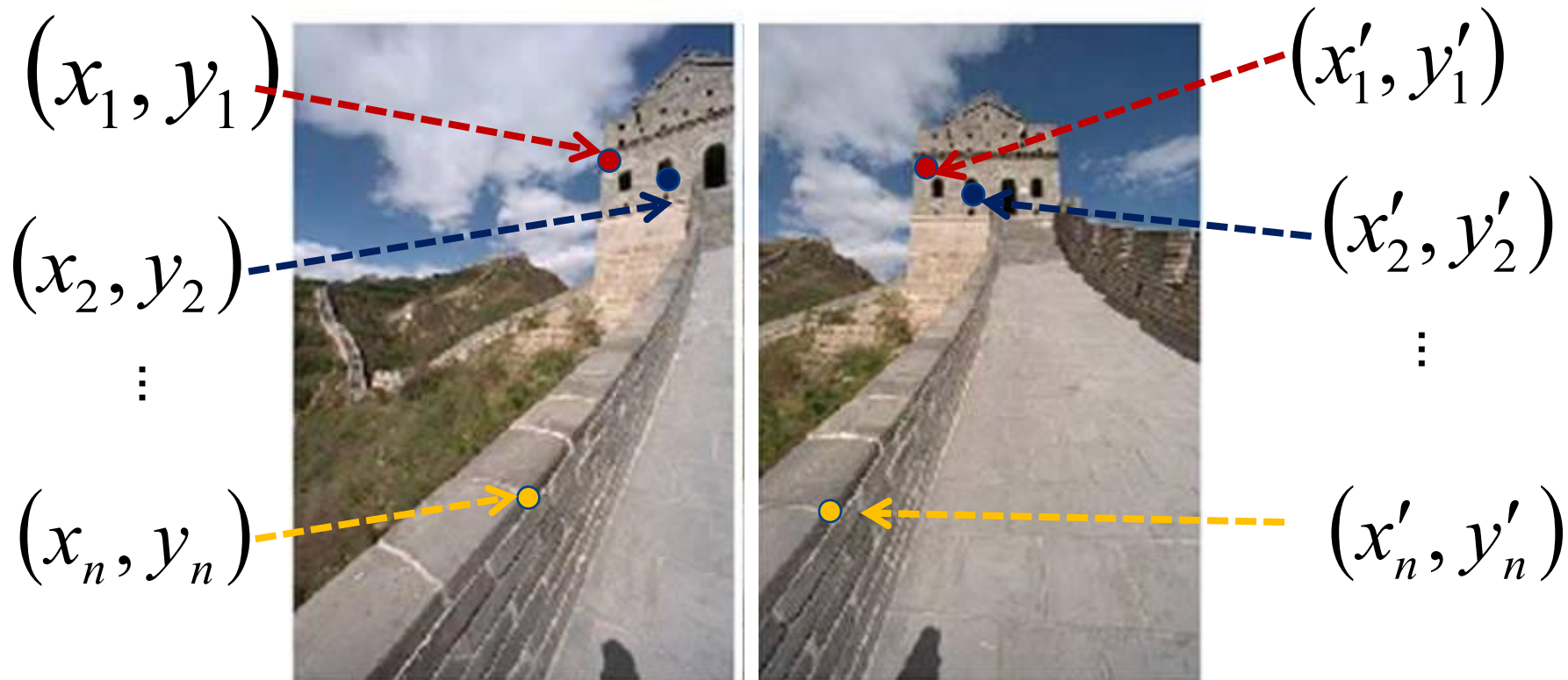
Chen Wang

Spatial AI & Robotics Lab

Department of Computer Science and Engineering

 **University at Buffalo** The State University of New York

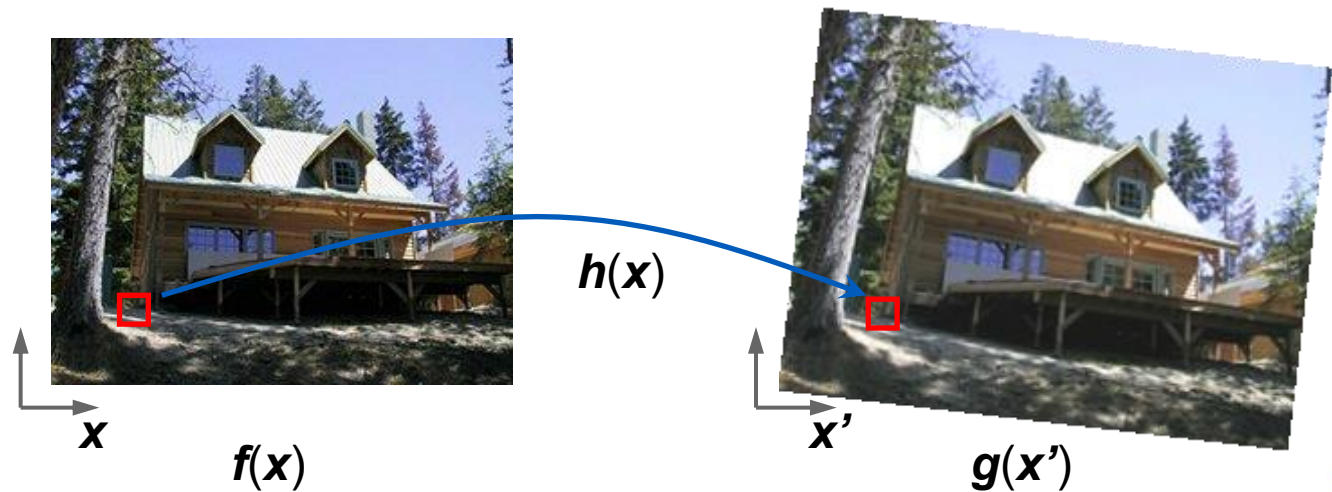
Recap: Alignment



How to stitch together a panorama?

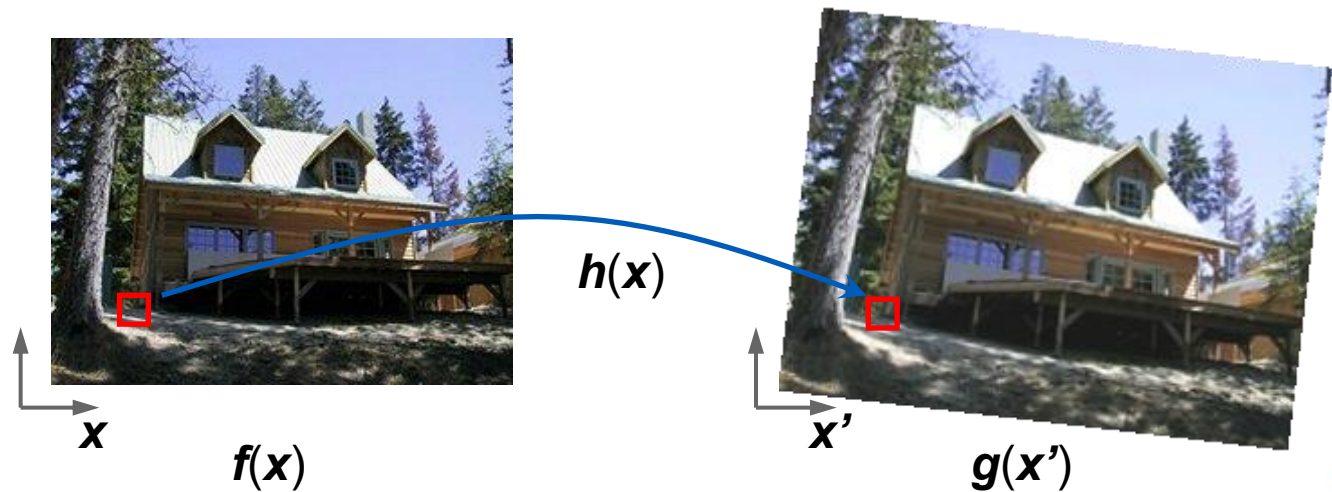
Image Warping

- Given a coordinate transform $\mathbf{x}' = \mathbf{h}(\mathbf{x})$ and a source image $\mathbf{f}(\mathbf{x})$, how do we compute a transformed image $\mathbf{g}(\mathbf{x}') = \mathbf{f}(\mathbf{h}(\mathbf{x}))$?



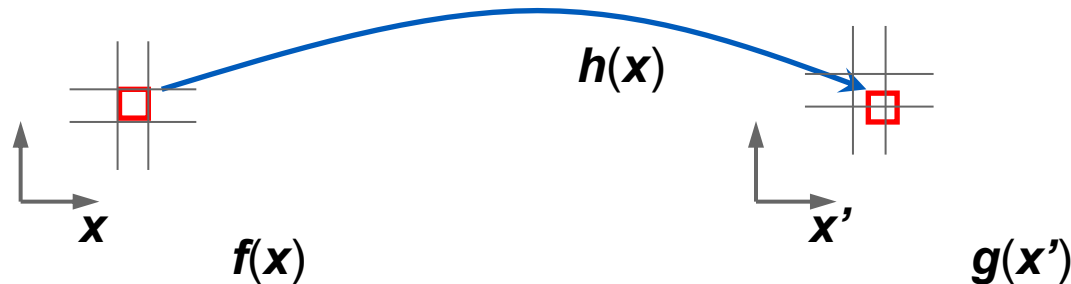
Forward Warping

- Send each pixel $f(\mathbf{x})$ to its corresponding location $\mathbf{x}' = h(\mathbf{x})$ in $g(\mathbf{x}')$
- What if pixel lands “between” two pixels?



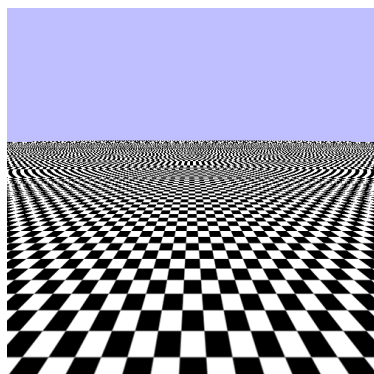
Forward Warping

- Send each pixel $f(\mathbf{x})$ to its corresponding location $\mathbf{x}' = h(\mathbf{x})$ in $g(\mathbf{x}')$
- What if pixel lands “between” two pixels?
- Answer: add “contribution” to several pixels, normalize later

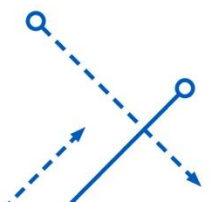
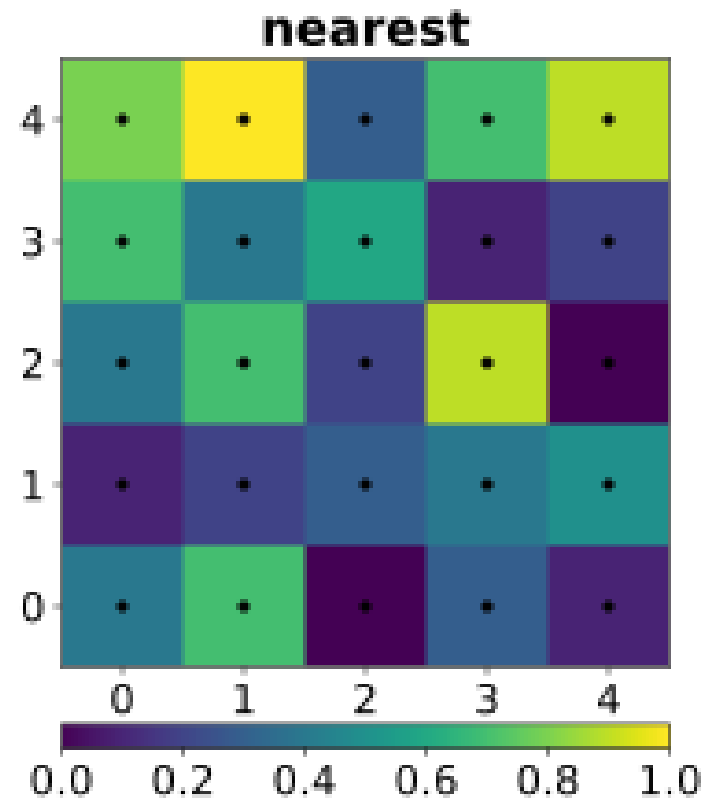
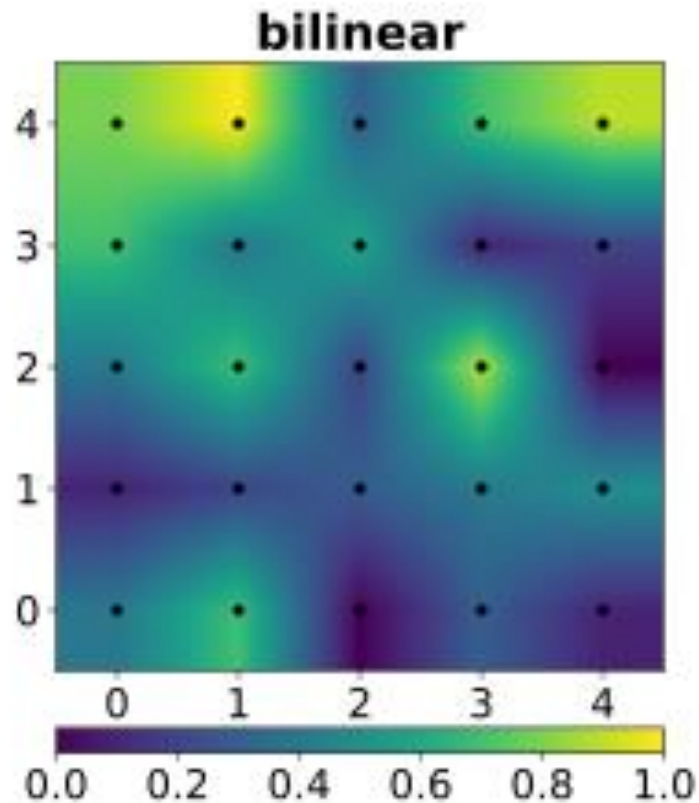


Interpolation

- Possible interpolation filters:
 - **Nearest Neighbor**
 - **Bilinear**
 - **Bicubic**
- Needed to prevent “jaggies” and “texture crawl”

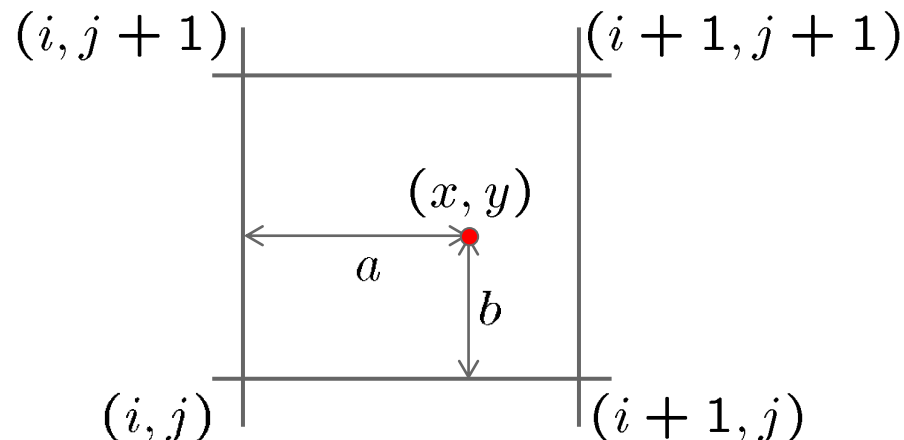


Bilinear interpolation



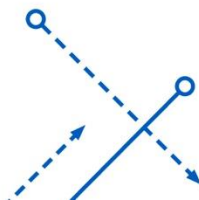
Bilinear interpolation

Sampling at $f(x,y)$:



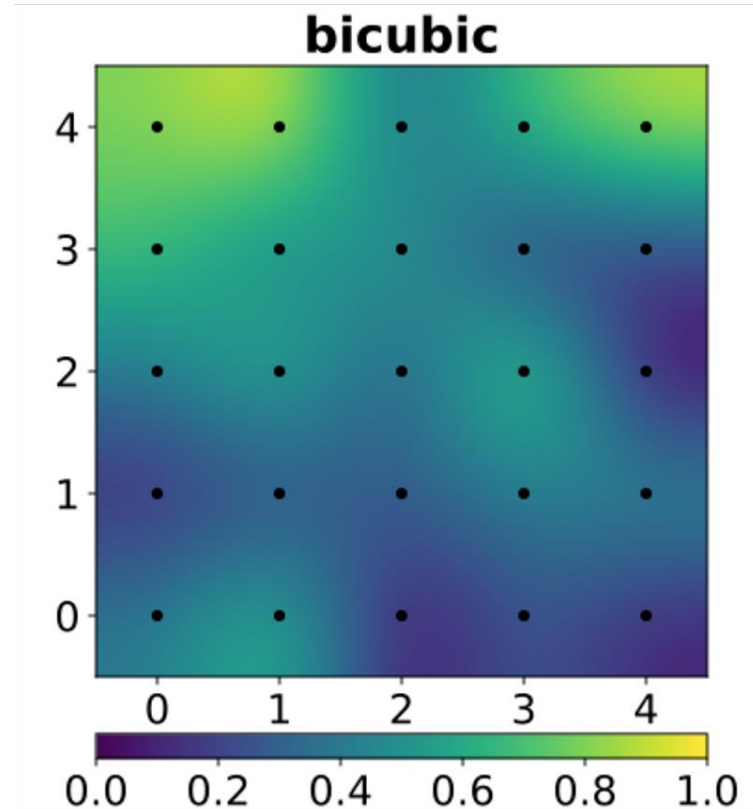
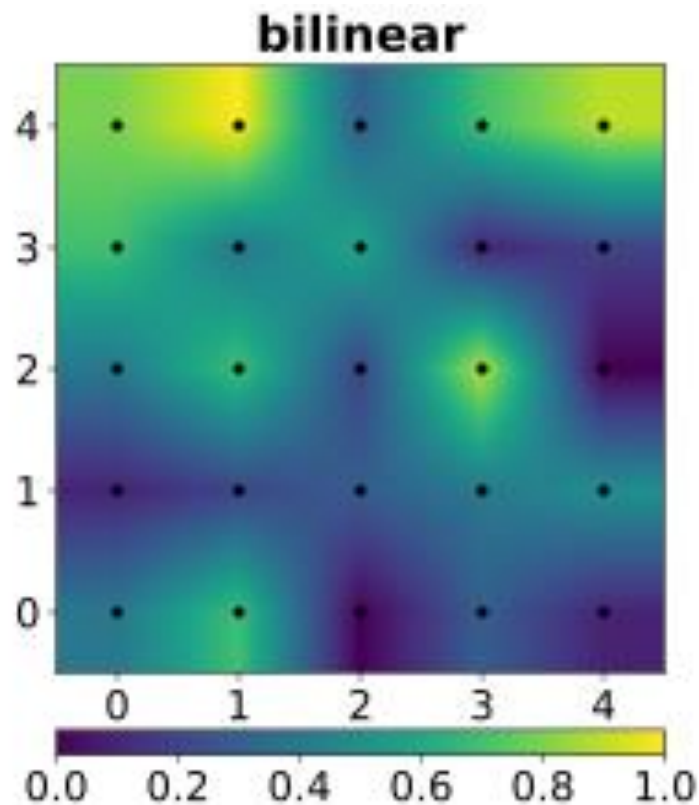
$$\begin{aligned} f(x, y) = & (1 - a)(1 - b) f[i, j] \\ & + a(1 - b) f[i + 1, j] \\ & + ab f[i + 1, j + 1] \\ & + (1 - a)b f[i, j + 1] \end{aligned}$$

Summation of Weights is 1



Bicubic interpolation

- Bilinear interpolation take **pixel intensities** into account.
- Bicubic interpolation **also** takes **image gradients** into account.



Bicubic interpolation

$$p(x, y) = \sum_{i=0}^3 \sum_{j=0}^3 a_{ij} x^i y^j.$$

surface $p(x, y)$ on the **unit square** $[0, 1] \times [0, 1]$ that is continuous

This requires determining the 16 coefficients.

Consider 4 corners of the unit square. $(0, 0)$ $(1, 0)$ $(0, 1)$ $(1, 1)$

1. $f(0, 0) = p(0, 0) = a_{00},$
2. $f(1, 0) = p(1, 0) = a_{00} + a_{10} + a_{20} + a_{30},$
3. $f(0, 1) = p(0, 1) = a_{00} + a_{01} + a_{02} + a_{03},$
4. $f(1, 1) = p(1, 1) = \sum_{i=0}^3 \sum_{j=0}^3 a_{ij}.$

Bicubic interpolation

We have following derivatives

$$p_x(x, y) = \sum_{i=1}^3 \sum_{j=0}^3 a_{ij} i x^{i-1} y^j,$$

$$p_y(x, y) = \sum_{i=0}^3 \sum_{j=1}^3 a_{ij} x^i j y^{j-1},$$

$$p_{xy}(x, y) = \sum_{i=1}^3 \sum_{j=1}^3 a_{ij} i x^{i-1} j y^{j-1}.$$

Likewise, eight equations for the derivatives in the x and the y directions:

1. $f_x(0, 0) = p_x(0, 0) = a_{10},$
2. $f_x(1, 0) = p_x(1, 0) = a_{10} + 2a_{20} + 3a_{30},$
3. $f_x(0, 1) = p_x(0, 1) = a_{10} + a_{11} + a_{12} + a_{13},$
4. $f_x(1, 1) = p_x(1, 1) = \sum_{i=1}^3 \sum_{j=0}^3 a_{ij} i,$
5. $f_y(0, 0) = p_y(0, 0) = a_{01},$
6. $f_y(1, 0) = p_y(1, 0) = a_{01} + a_{11} + a_{21} + a_{31},$
7. $f_y(0, 1) = p_y(0, 1) = a_{01} + 2a_{02} + 3a_{03},$
8. $f_y(1, 1) = p_y(1, 1) = \sum_{i=0}^3 \sum_{j=1}^3 a_{ij} j.$

And four equations for the xy mixed partial derivative:

1. $f_{xy}(0, 0) = p_{xy}(0, 0) = a_{11},$
2. $f_{xy}(1, 0) = p_{xy}(1, 0) = a_{11} + 2a_{21} + 3a_{31},$
3. $f_{xy}(0, 1) = p_{xy}(0, 1) = a_{11} + 2a_{12} + 3a_{13},$
4. $f_{xy}(1, 1) = p_{xy}(1, 1) = \sum_{i=1}^3 \sum_{j=1}^3 a_{ij} ij.$

Bicubic interpolation

Grouping the unknown parameters a_{ij} in a vector

$$\alpha = [a_{00} \ a_{10} \ a_{20} \ a_{30} \ a_{01} \ a_{11} \ a_{21} \ a_{31} \ a_{02} \ a_{12} \ a_{22} \ a_{32} \ a_{03} \ a_{13} \ a_{23} \ a_{33}]^T$$

and letting

$$x = [f(0,0) \ f(1,0) \ f(0,1) \ f(1,1) \ f_x(0,0) \ f_x(1,0) \ f_x(0,1) \ f_x(1,1) \ f_y(0,0) \ f_y(1,0) \ f_y(0,1) \ f_y(1,1) \ f_{xy}(0,0) \ f_{xy}(1,0) \ f_{xy}(0,1) \ f_{xy}(1,1)]^T,$$

the above system of equations can be reformulated into a matrix for the linear equation $A\alpha = x$.

Inverting the matrix gives the more useful linear equation $A^{-1}x = \alpha$, where

$$A^{-1} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 3 & 0 & 0 & -2 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 2 & -2 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -3 & 3 & 0 & 0 & -2 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 2 & -2 & 0 & 0 & 1 & 1 & 0 & 0 \\ -3 & 0 & 3 & 0 & 0 & 0 & 0 & 0 & -2 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -3 & 0 & 3 & 0 & 0 & 0 & 0 & 0 & -2 & 0 & -1 & 0 \\ 9 & -9 & -9 & 9 & 6 & 3 & -6 & -3 & 6 & -6 & 3 & -3 & 4 & 2 & 2 & 1 \\ -6 & 6 & 6 & -6 & -3 & -3 & 3 & 3 & -4 & 4 & -2 & 2 & -2 & -2 & -1 & -1 \\ 2 & 0 & -2 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 2 & 0 & -2 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ -6 & 6 & 6 & -6 & -4 & -2 & 4 & 2 & -3 & 3 & -3 & 3 & -2 & -1 & -2 & -1 \\ 4 & -4 & -4 & 4 & 2 & 2 & -2 & -2 & 2 & -2 & 2 & -2 & 1 & 1 & 1 & 1 \end{bmatrix},$$

which allows α to be calculated quickly and easily.

A more compact form:

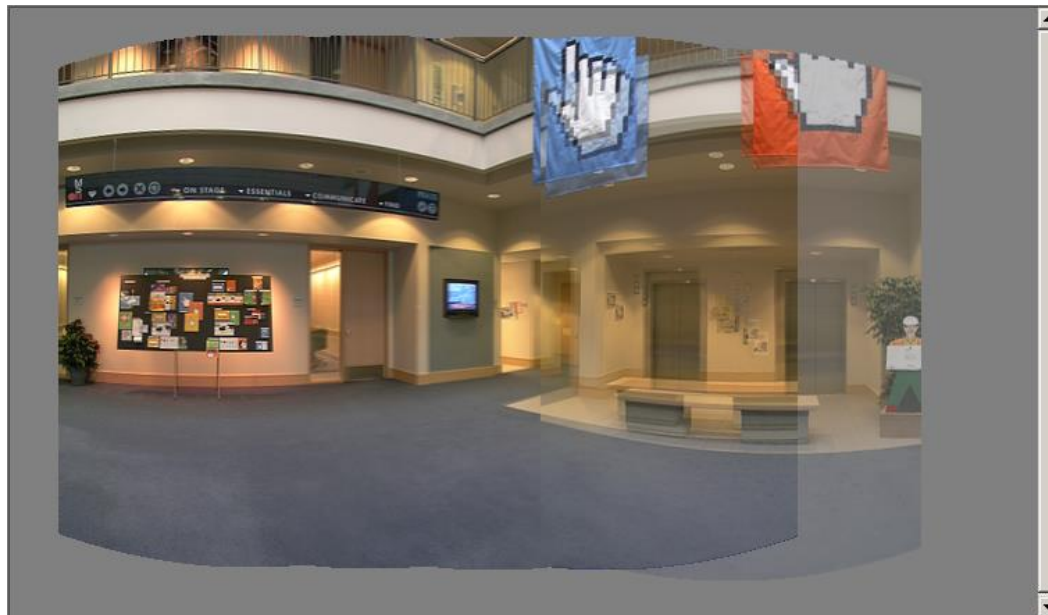
$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -3 & 3 & -2 & -1 \\ 2 & -2 & 1 & 1 \end{bmatrix} \begin{bmatrix} f(0,0) & f(0,1) & f_y(0,0) & f_y(0,1) \\ f(1,0) & f(1,1) & f_y(1,0) & f_y(1,1) \\ f_x(0,0) & f_x(0,1) & f_{xy}(0,0) & f_{xy}(0,1) \\ f_x(1,0) & f_x(1,1) & f_{xy}(1,0) & f_{xy}(1,1) \end{bmatrix} \begin{bmatrix} 1 & 0 & -3 & 2 \\ 0 & 0 & 3 & -2 \\ 0 & 1 & -2 & 1 \\ 0 & 0 & -1 & 1 \end{bmatrix},$$

Content

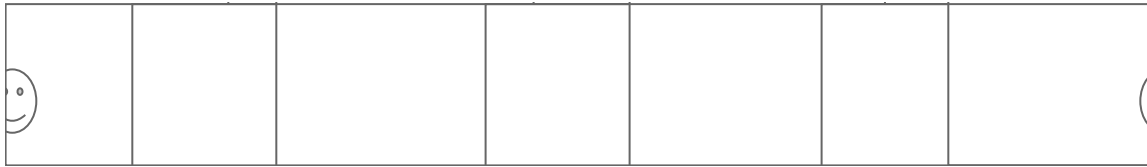
- Stitching (Blending)
 - Panorama, Blending, Deghosting
 - Feathering, Pyramid Blending
 - Laplacian Blending

Image Stitching

1. Align the images over each other
 - camera pan $\leftrightarrow$ translation on cylinder
2. Blend the images together

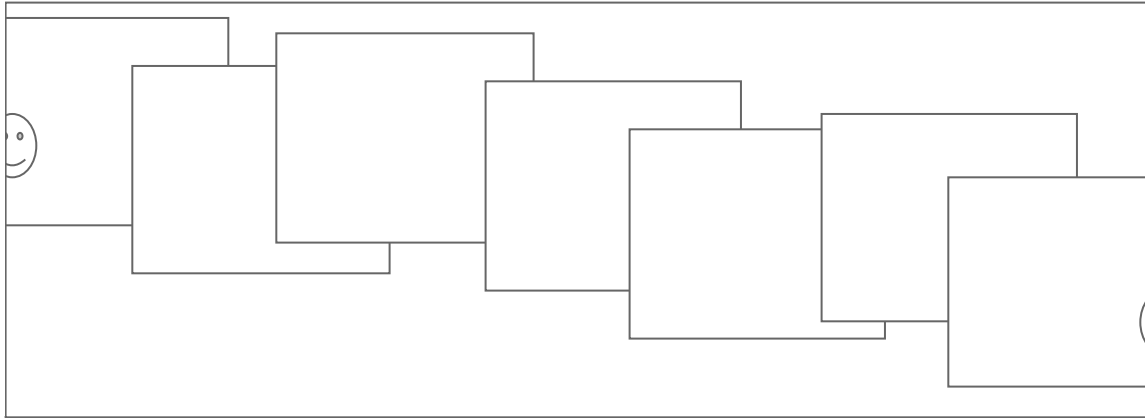


Assembling the panorama



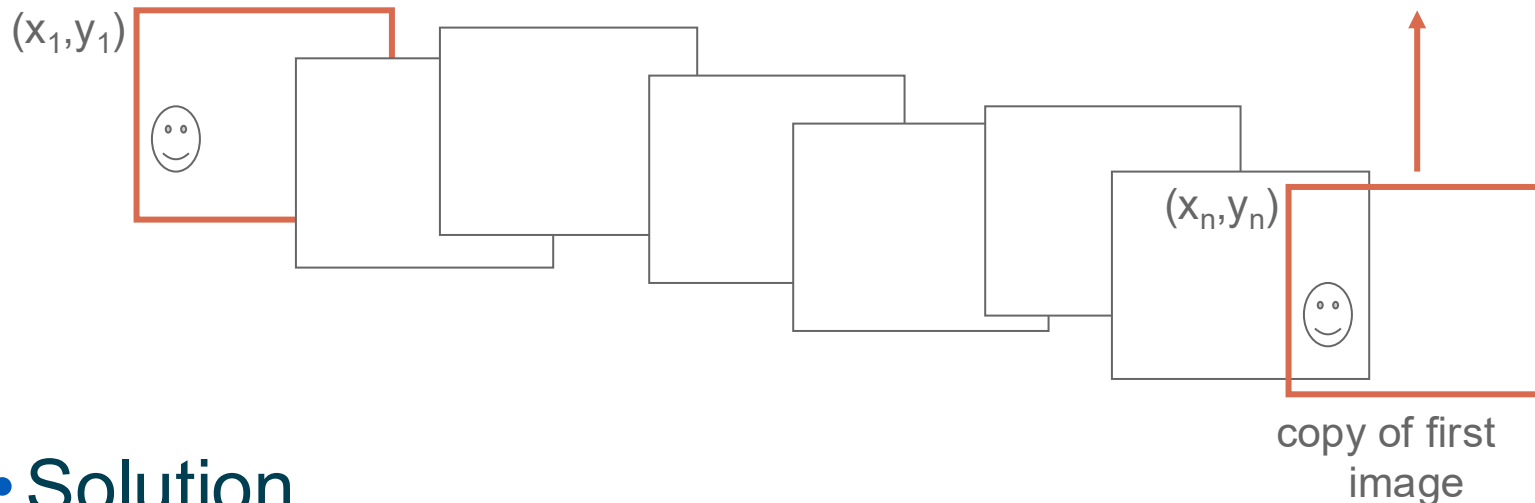
- Stitch pairs together, blend, then crop

Problem: Drift



- Error accumulation
 - small (vertical) errors accumulate over time
 - apply correction so that $\text{sum} = 0$ (for 360° pan.)

Problem: Drift



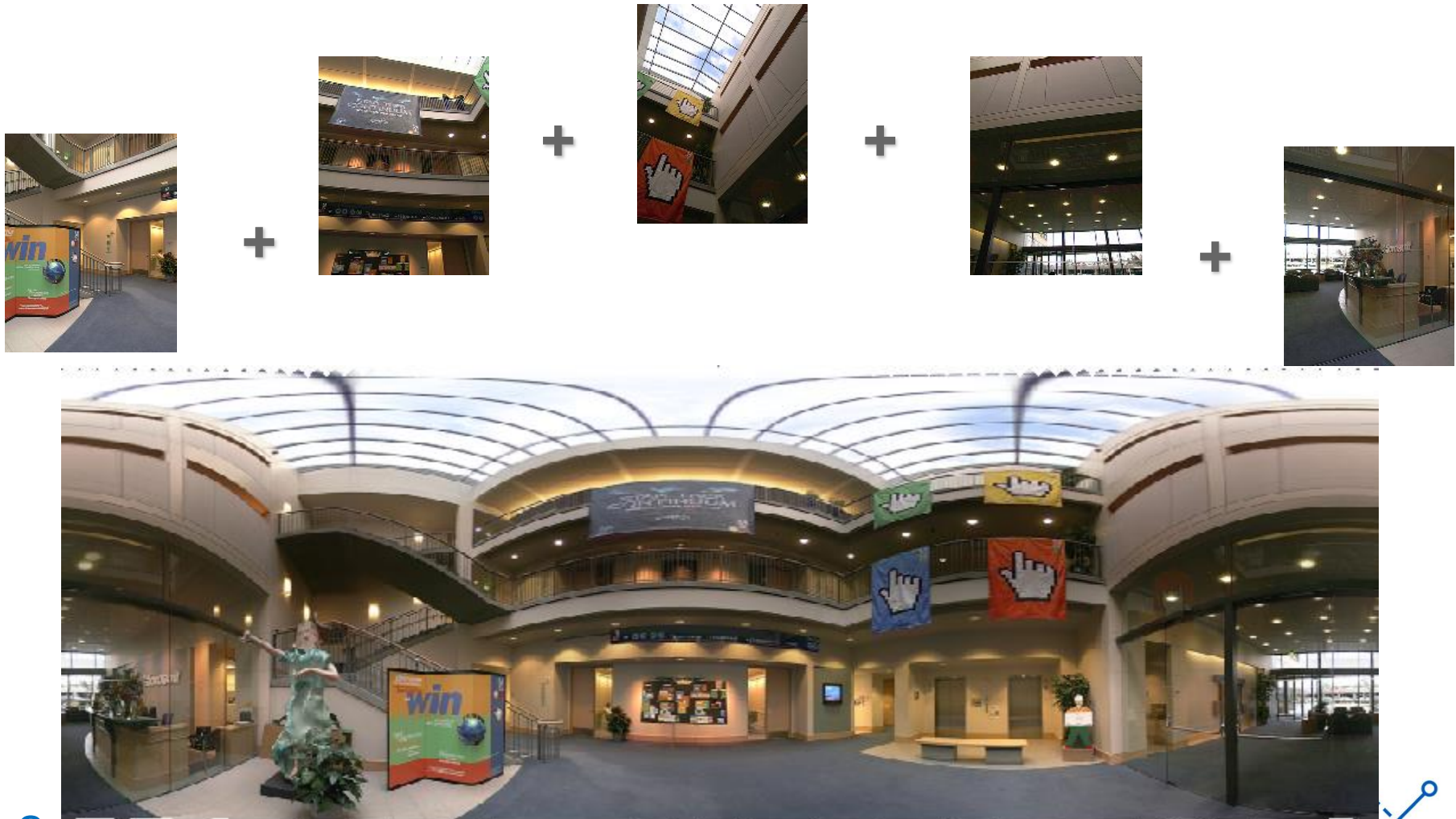
• Solution

- add another copy of first image at the end
- this gives a constraint: $y_n = y_1$
- there are a bunch of ways to solve this problem
 - add displacement of $(y_1 - y_n)/(n - 1)$ to each image after the first
 - compute a global warp: $y' = y + ax$
 - run a big optimization problem, incorporating this constraint
 - best solution, but more complicated.

Full-view (360° spherical) panoramas



Full-view Panorama

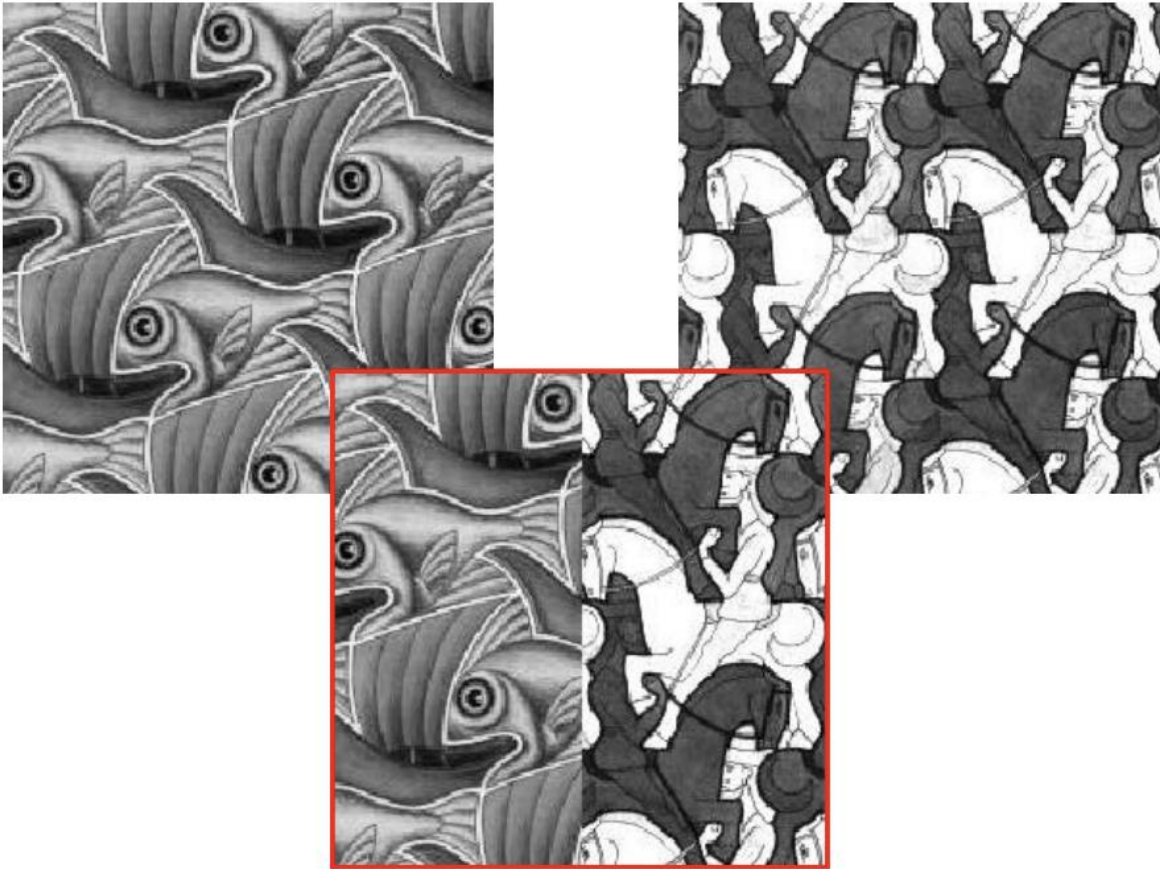


Texture Mapped Model

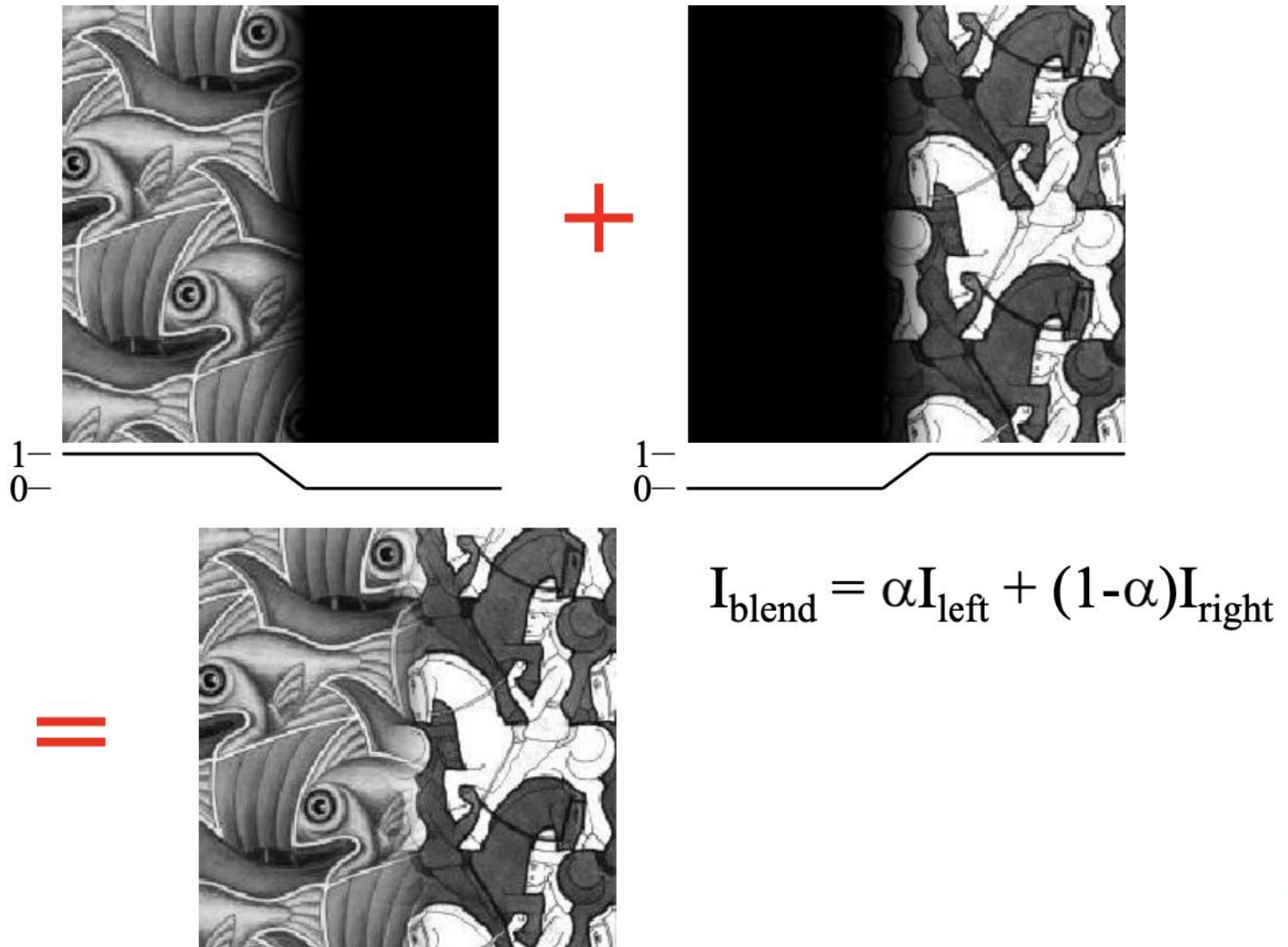


Image Blending

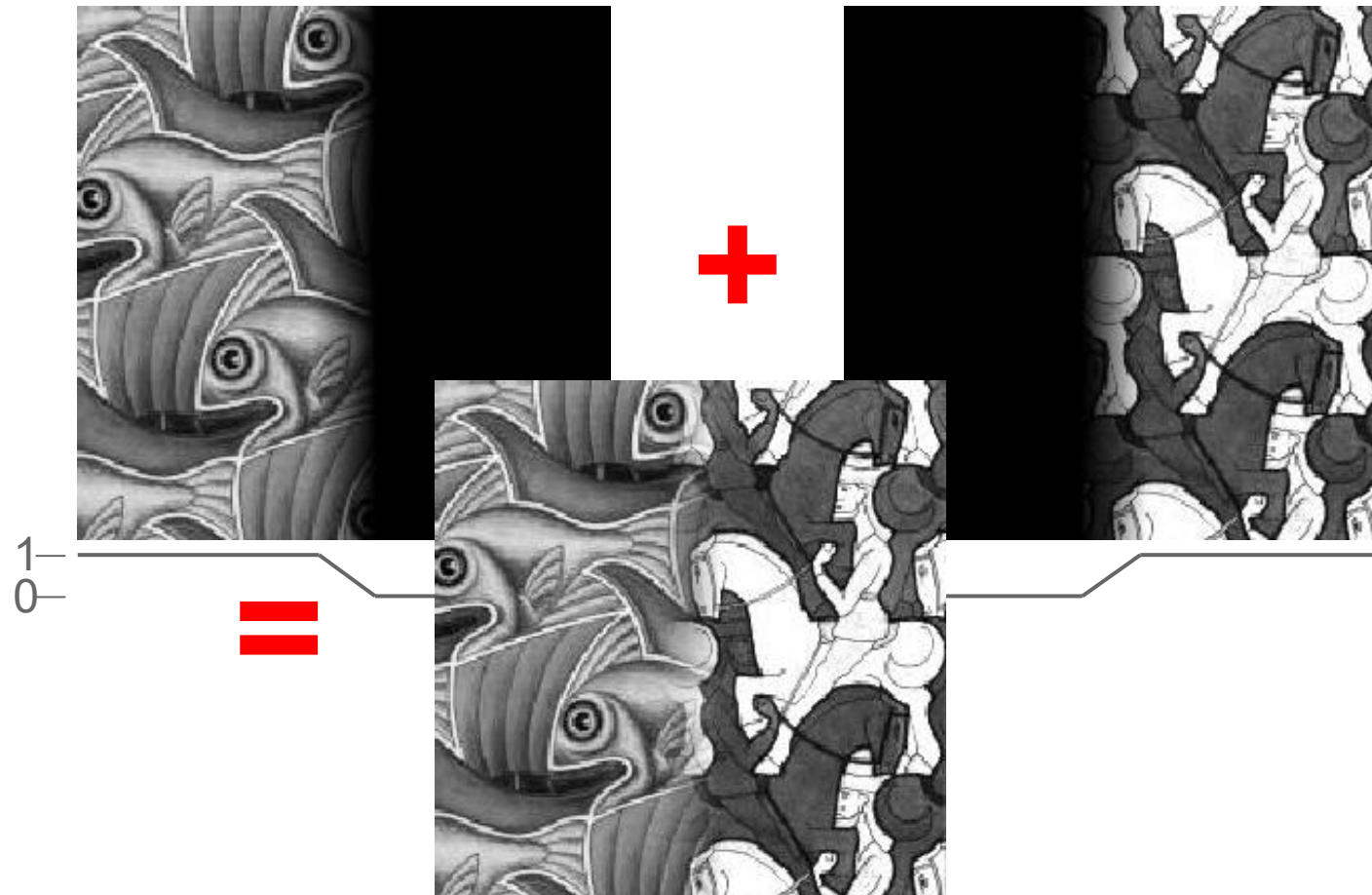
- How do we put images together so there is no obvious divisions?



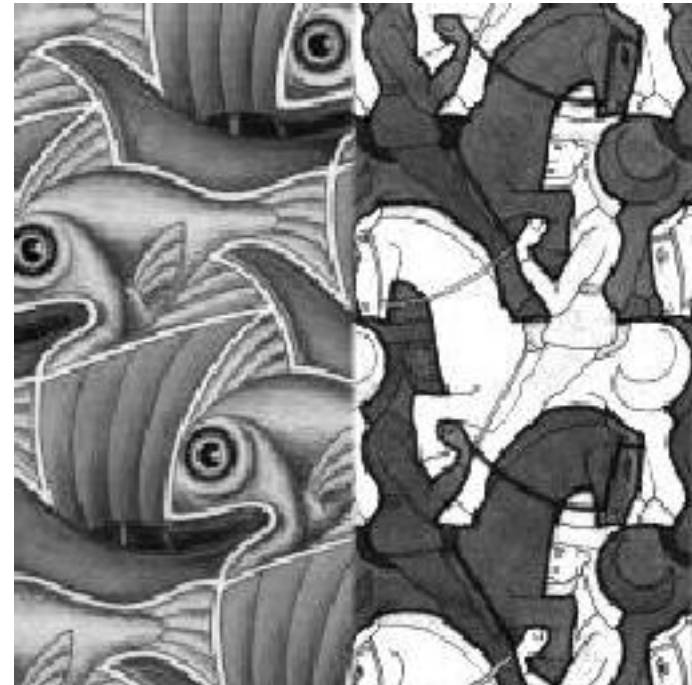
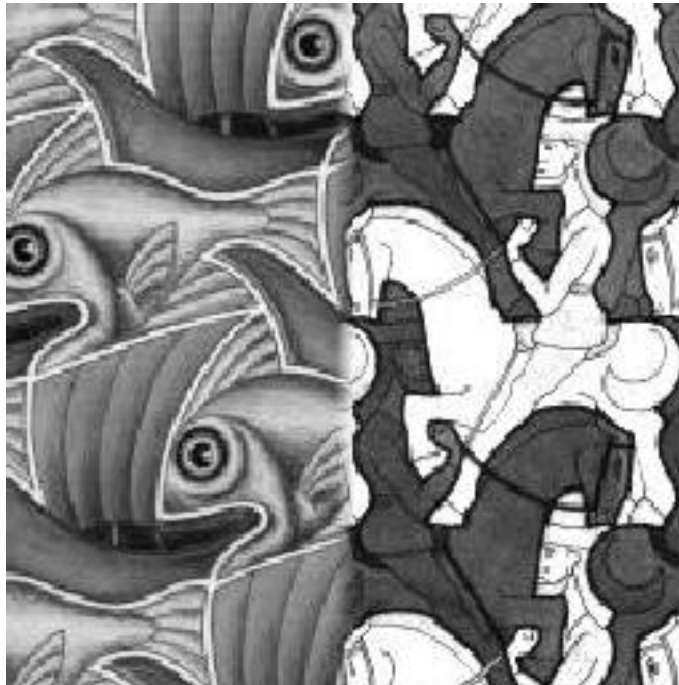
Alpha Blending / Feathering



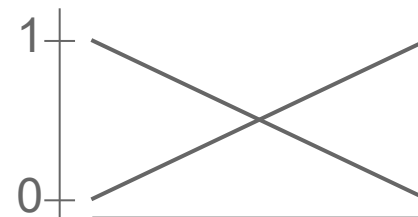
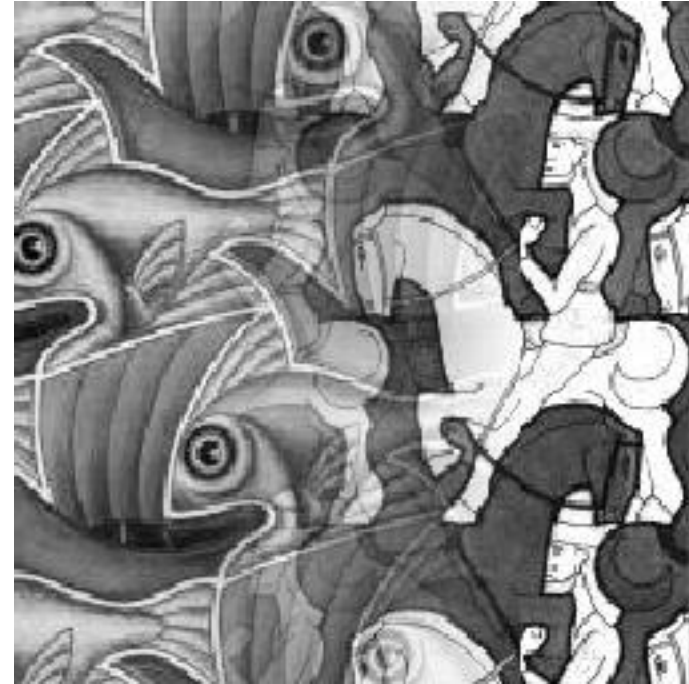
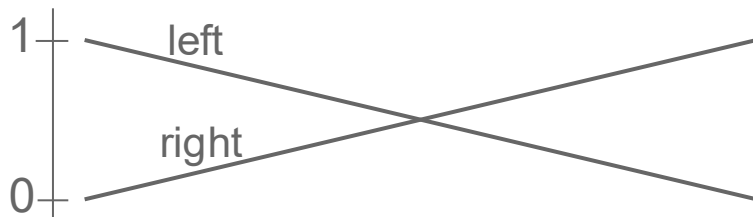
Feathering



Effect of window size

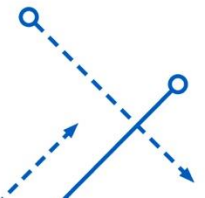


Effect of window size



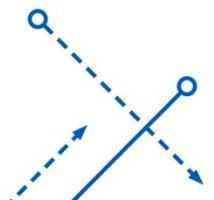
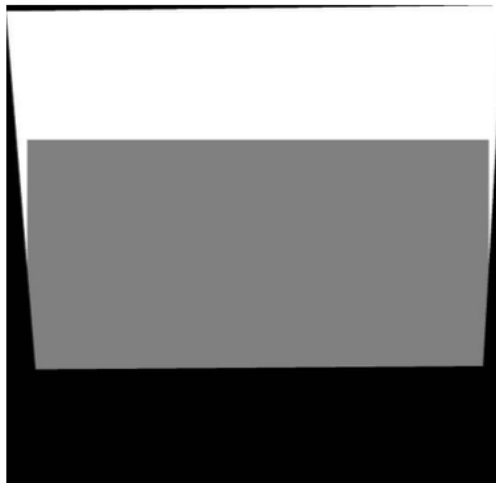
Good Window Size

- “Optimal” Window: smooth but not ghosted.
- To avoid seams
 - window $\geq$ size of largest prominent feature
- To avoid ghosting
 - window $\leq 2 \times$ size of smallest prominent feature



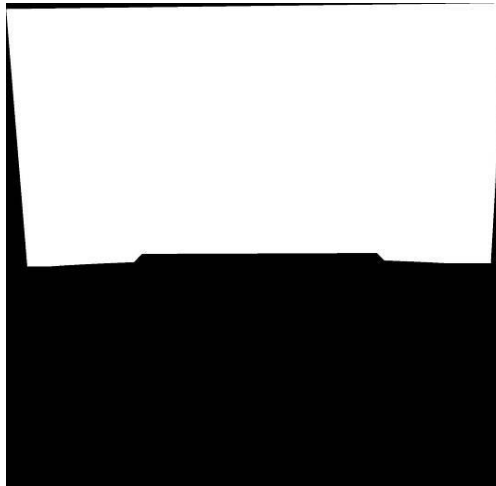
Setting alpha: simple averaging

- Alpha = 0.5 in the overlap region



Setting alpha: center seam

- Alpha = $\text{logical}(\text{dtrans1} > \text{dtrans2})$



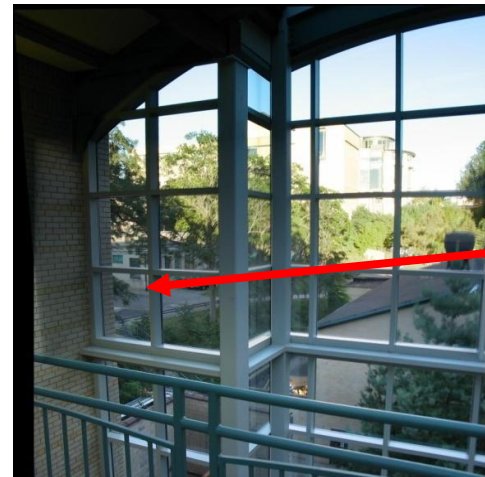
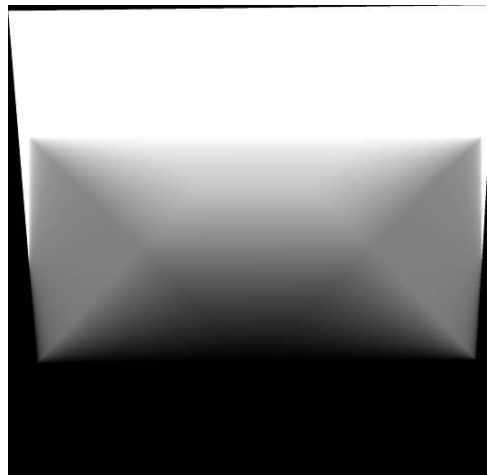
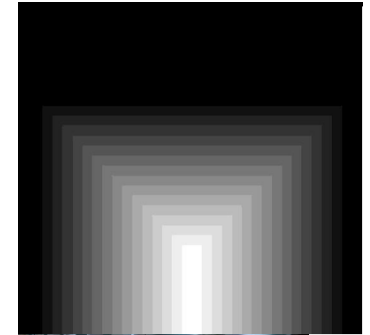
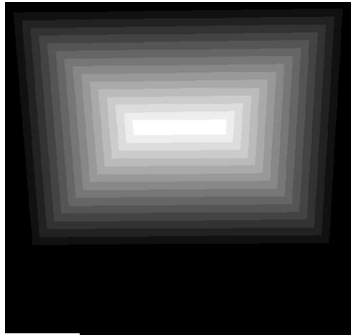
Setting alpha: blurred seam

- Alpha = blurred

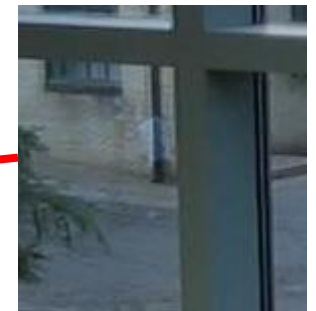


Setting alpha: center weighting

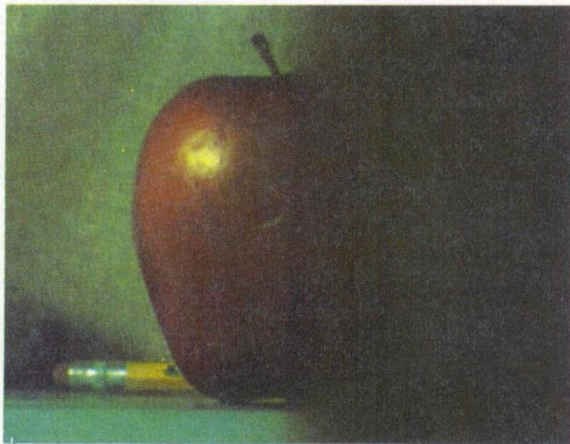
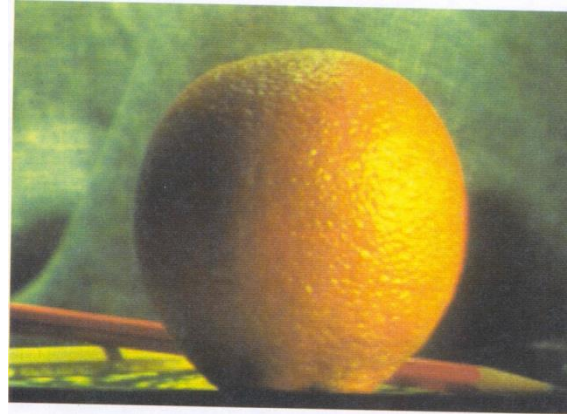
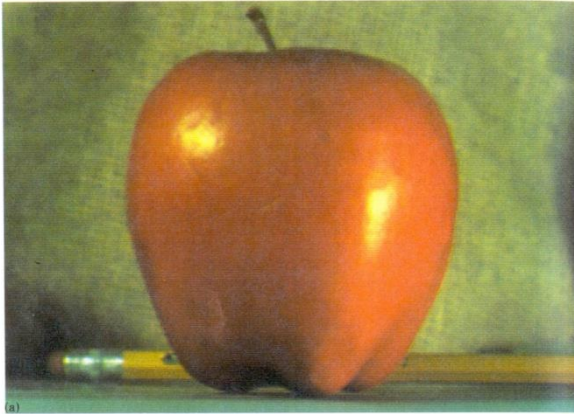
- Alpha = $d_{trans1} / (d_{trans1} + d_{trans2})$



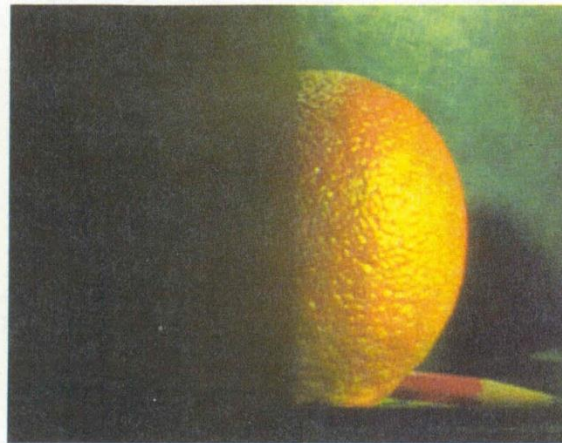
Ghost



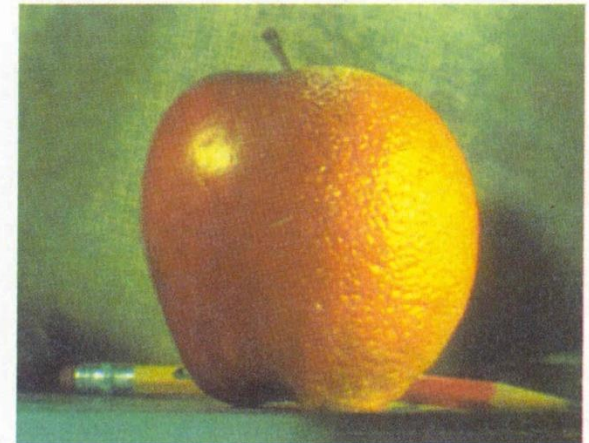
Pyramid Blending



(d)



(h)

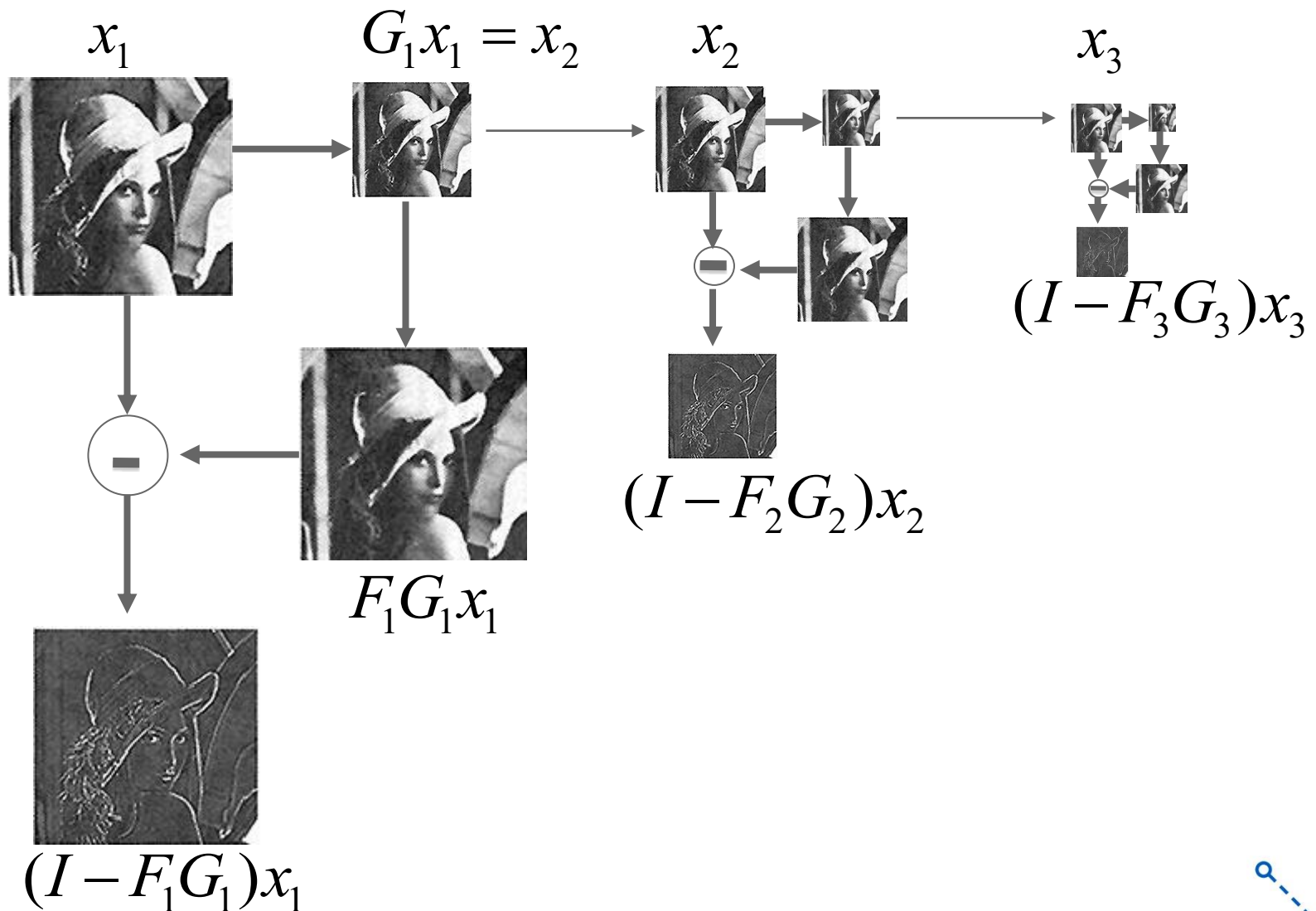


(l)

Laplacian Pyramid (Recap)

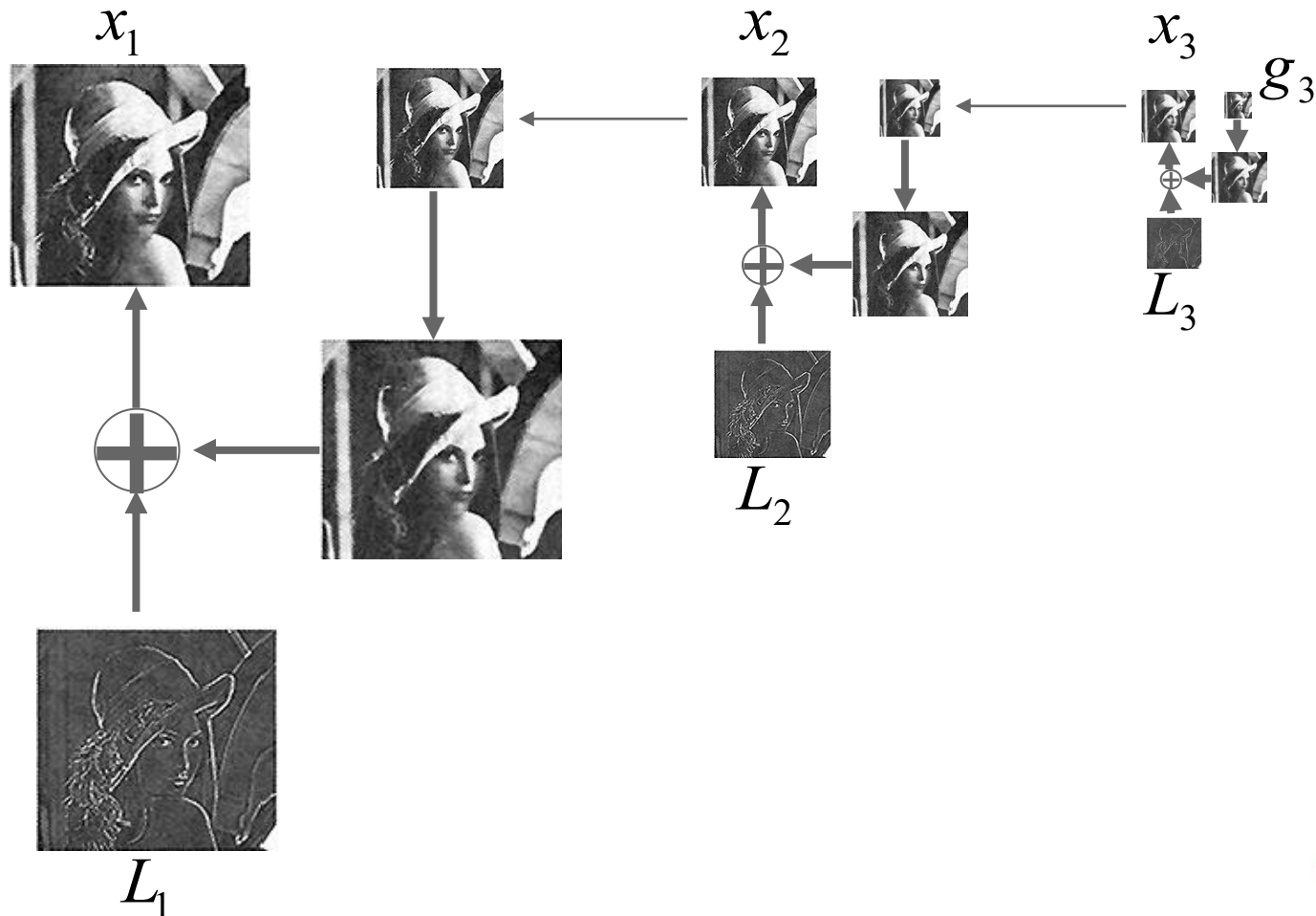
- **Difference between up-sampled Gaussian pyramid and Gaussian pyramid.**
- **Band pass filter** - each level represents spatial frequencies (largely) unrepresented at other level.

Laplacian pyramid (Recap)



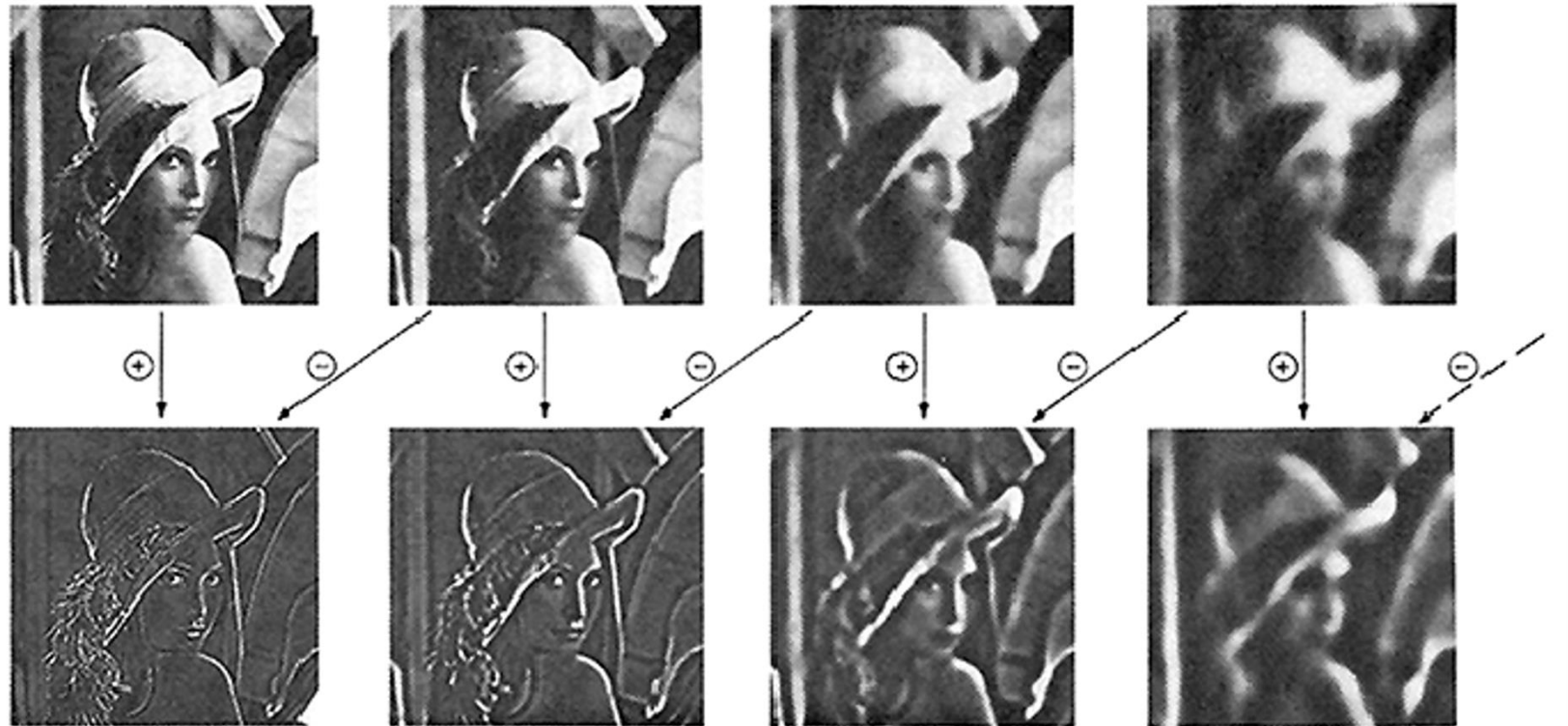
Laplacian Pyramid (Recap)

- Reconstruction: recover x_1 from L_1, L_2, L_3 and g_3

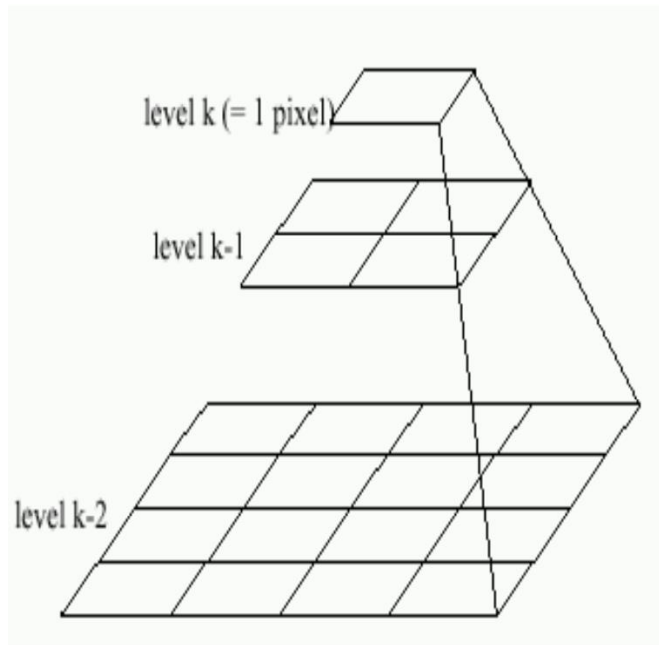


Laplacian Pyramid (Recap)

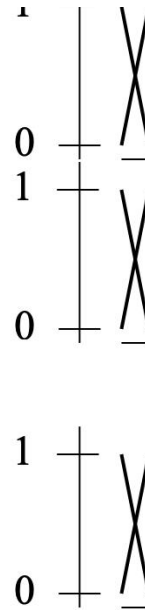
- Information captured at each level of a Gaussian (top) and Laplacian (bottom) pyramid
 - showing full resolution.



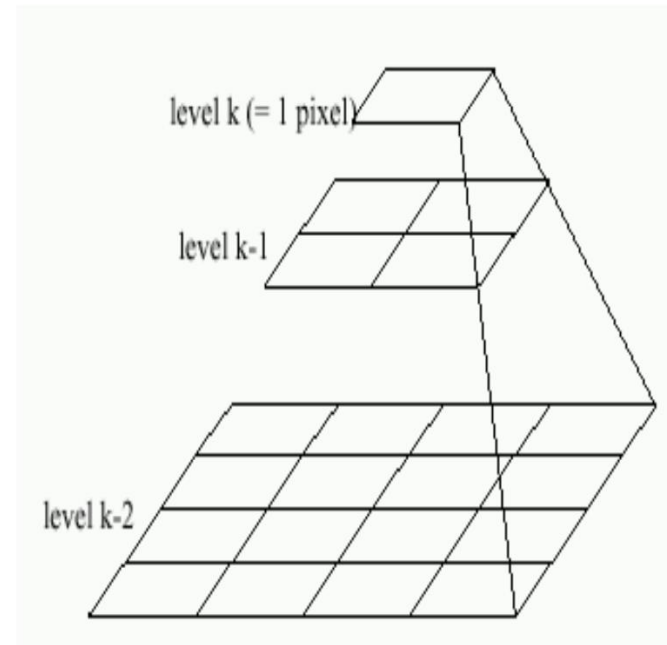
Pyramid Blending



Left pyramid

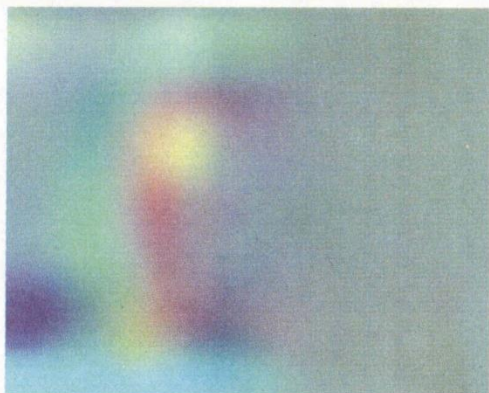


blend

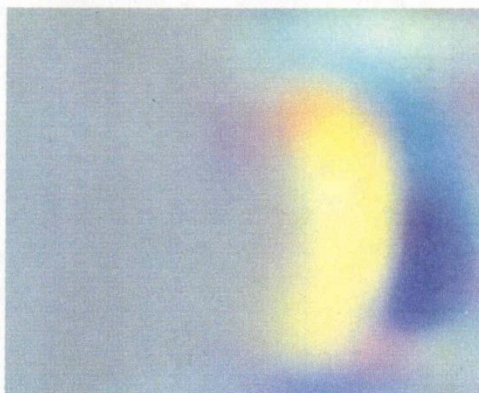


Right pyramid

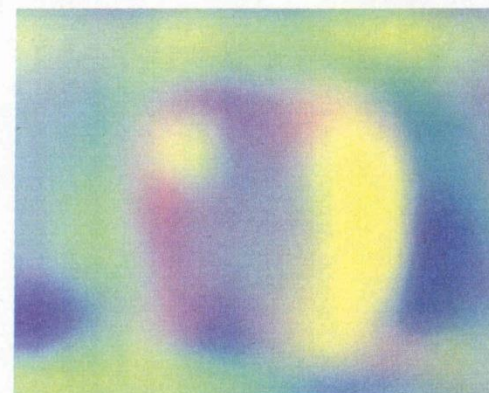
Laplacian
level
4



(c)

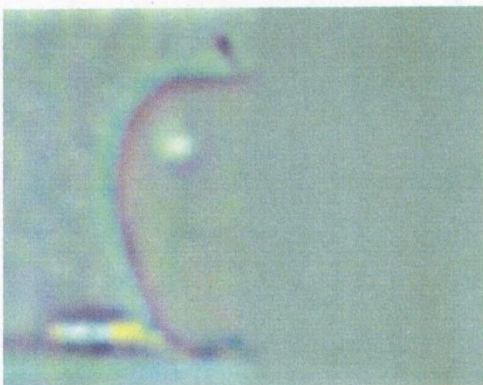


(g)

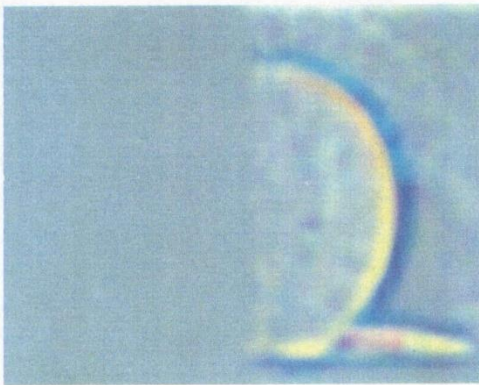


(k)

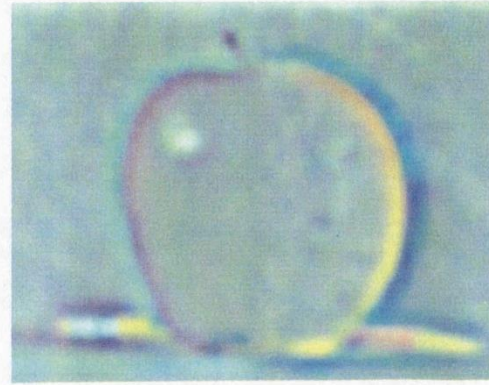
Laplacian
level
2



(b)

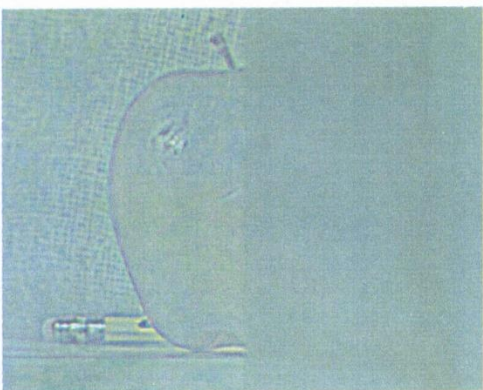


(f)

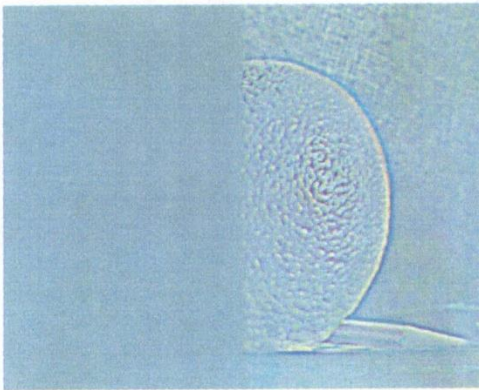


(j)

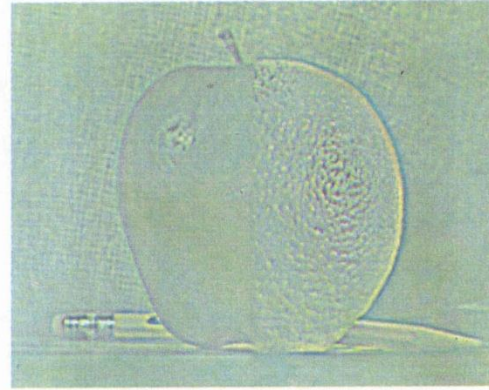
Laplacian
level
0



(a)



(e)



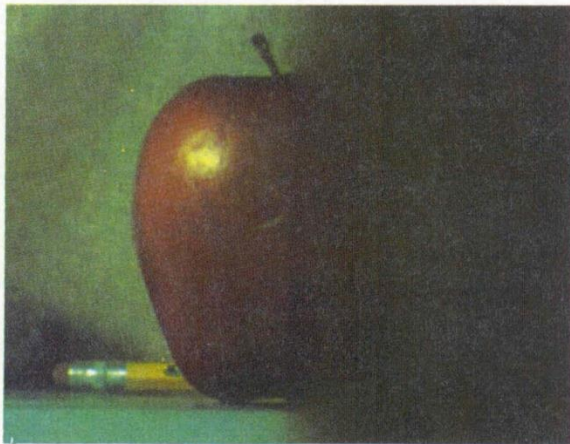
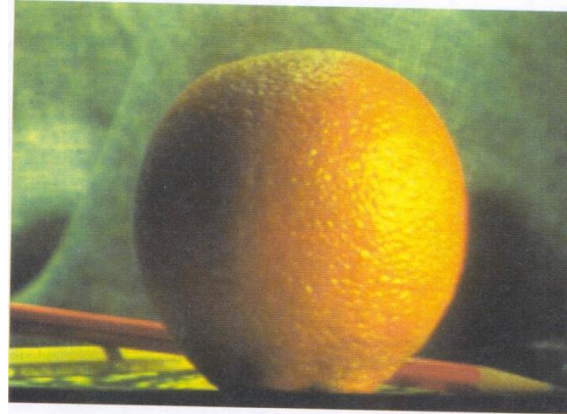
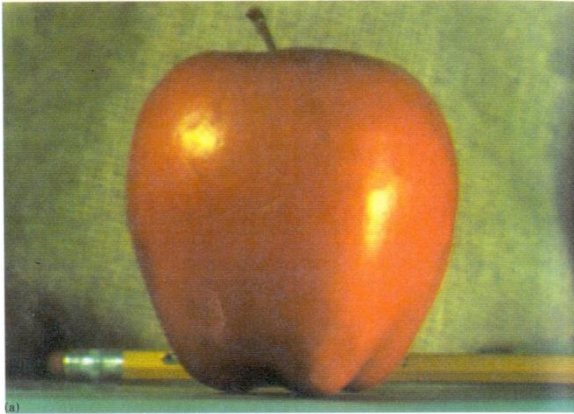
(i)

left pyramid

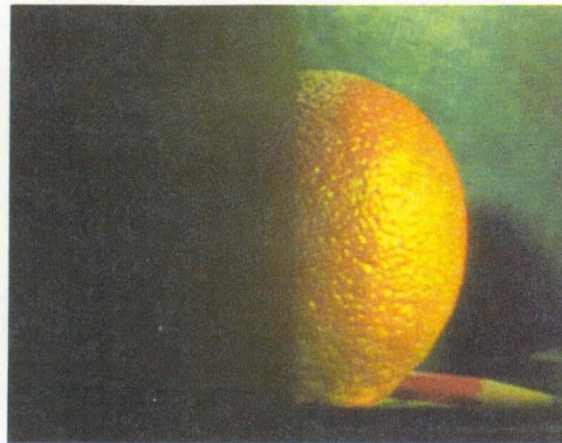
right pyramid

blended pyramid

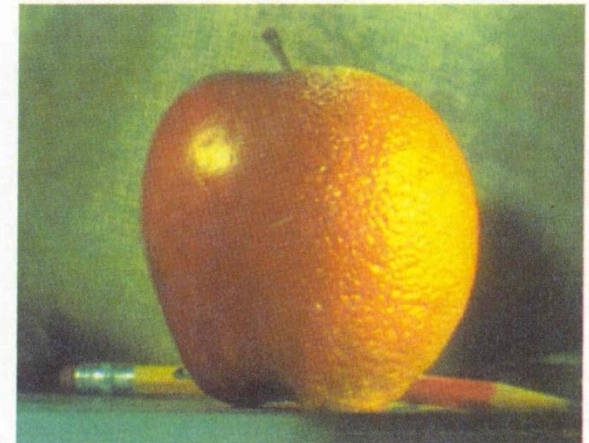
Pyramid Blending



(d)



(h)

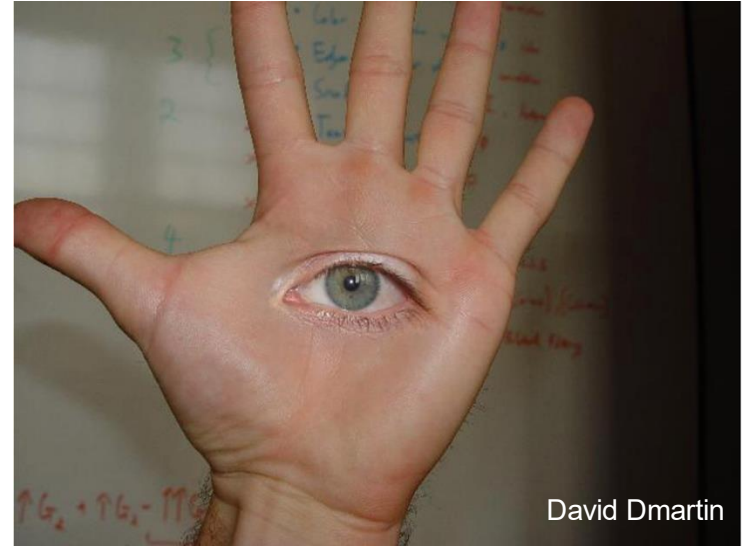
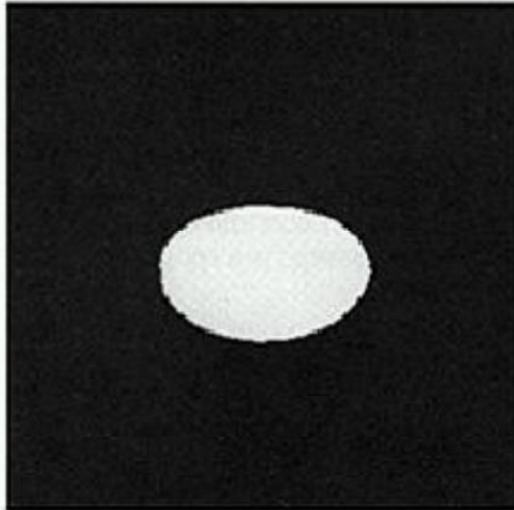


(l)

Laplacian Pyramid: Blending

- 1. Build Laplacian pyramids LA and LB from images A and B
- 2. Build a Gaussian pyramid GR from selected region R
 - E.g., Gaussian pyramid from a mask R :
 - 1: take this pixel/region from **image A**.
 - 0: take this pixel/region from **image B**.
- 3. Combined pyramid LS from LA / LB using GR as weights:
 - $LS(i, j) = GR(i, j) * LA(i, j) + (1 - GR(i, j)) * LB(i, j)$
- 4. Collapse the LS pyramid to get the final blended image

Blending Regions



David Dmartin



Chris Cameron

Season Blending



Don't blend, CUT!

- So far we only tried to blend between two images.
- What about finding an optimal seam?



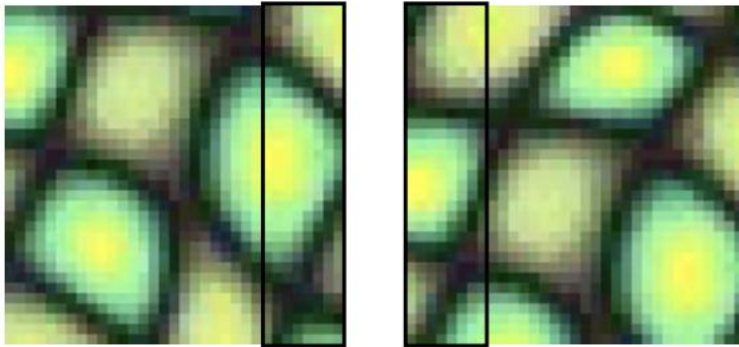
Don't blend, CUT!

- Segment the mosaic
 - Single source image per segment
- Avoid artifacts along boundaries
 - Dijkstra's algorithm

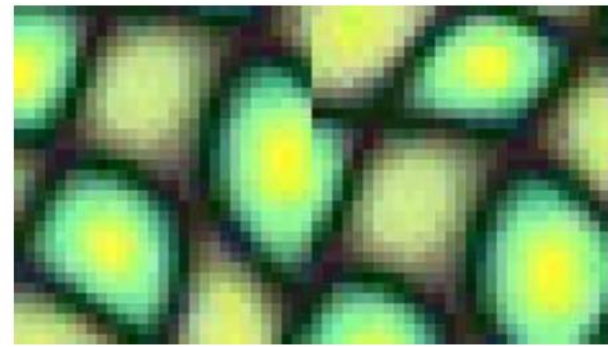


Minimal error boundary

overlapping blocks

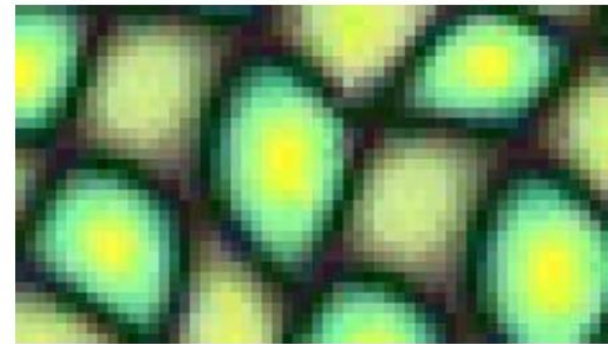


vertical boundary



$$\left[\begin{array}{c} \text{block 1} \\ - \\ \text{block 2} \end{array} \right]^2 = \text{error map}$$
The diagram shows two vertical blocks of the cell image, one above the other, with a minus sign between them. This is enclosed in large square brackets, followed by a superscript 2. An equals sign follows, leading to a vertical strip of the image where the overlap error is highlighted in red.

overlap error



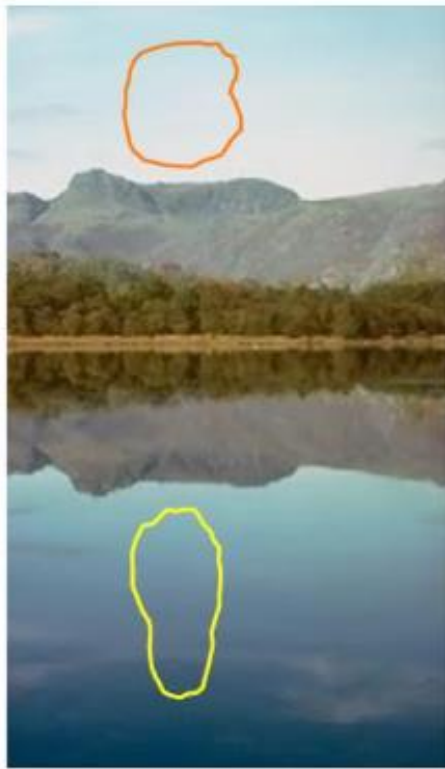
min. error boundary

Gradient Domain Blending

- Blend the gradients of the two images, then integrate.
 - Focuses on **how colors change** across the image (the gradients) rather than the absolute colors themselves. By matching or modifying these gradients, the final composite looks smoother and more natural.



sources



destinations



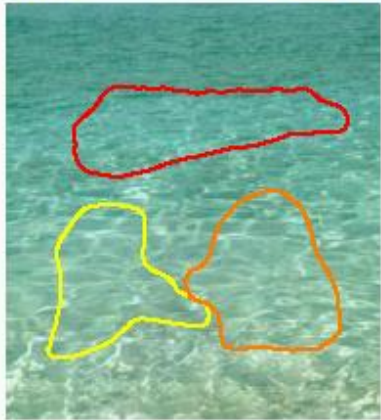
cloning



seamless cloning

Gradient Domain Blending

- Blend the gradients of the two images, then integrate.



sources/destinations



cloning



seamless cloning

Video (Gradient Domain Fusion)

Interactive Digital Photomontage

Aseem Agarwala, Mira Dontcheva
Maneesh Agrawala, Steven Drucker, Alex Colburn
Brian Curless, David Salesin, Michael Cohen



Final thought: What is a “panorama”?

- Tracking a subject
- Repeated (best) shots
- Multiple exposures
- “Infer” what photographer wants?
- Also referred to as “Computational Photography”



Project 2: Stitching

- Feature Detection
- Feature Matching
- Homography
- RANSAC
- Global alignment
- Warping
- Blending



(a)



(b)

